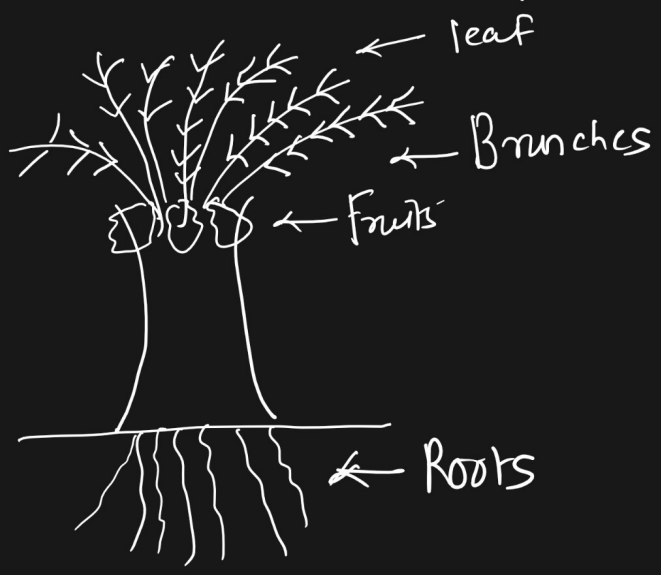
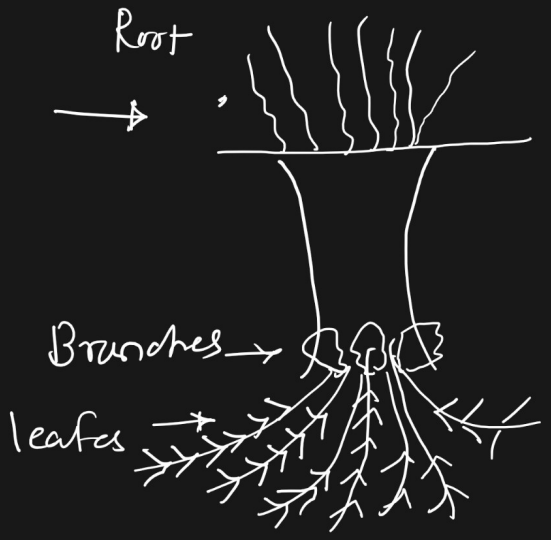
Day No. 23 Tree





Tree :-

⇒ It is a nonlinear data structures

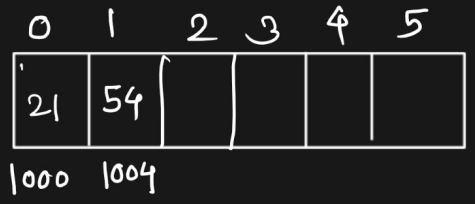
⇒ On top we have root node & root nodes they have child nodes

⇒ Nodes are connected in Hirarchial fashion (we store the data in Hirarchy Fashion)

⇒ One node may be connected to the multiple child nodes.

Linear DS

1) Array :-

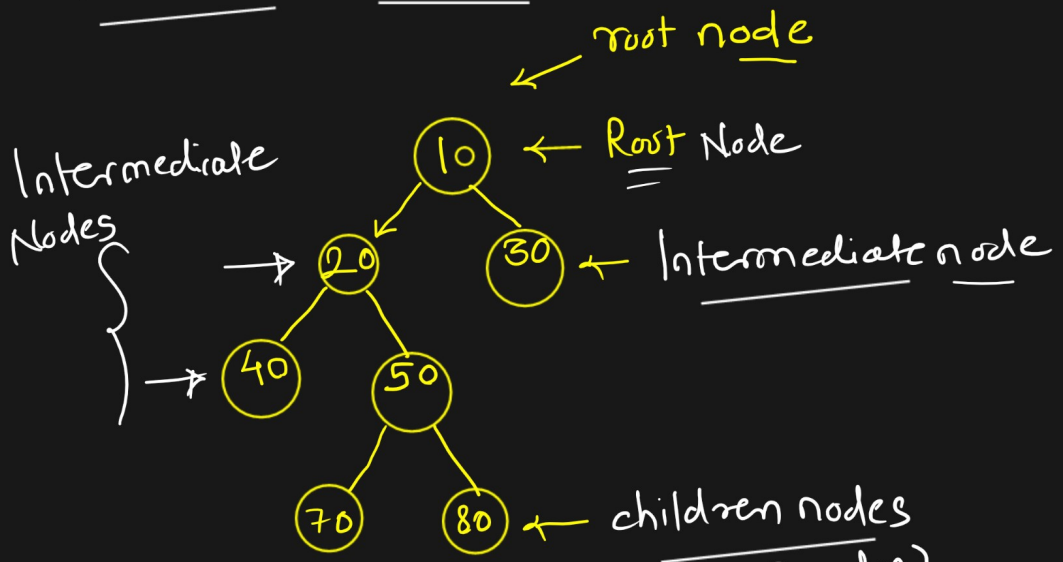


2) Stack

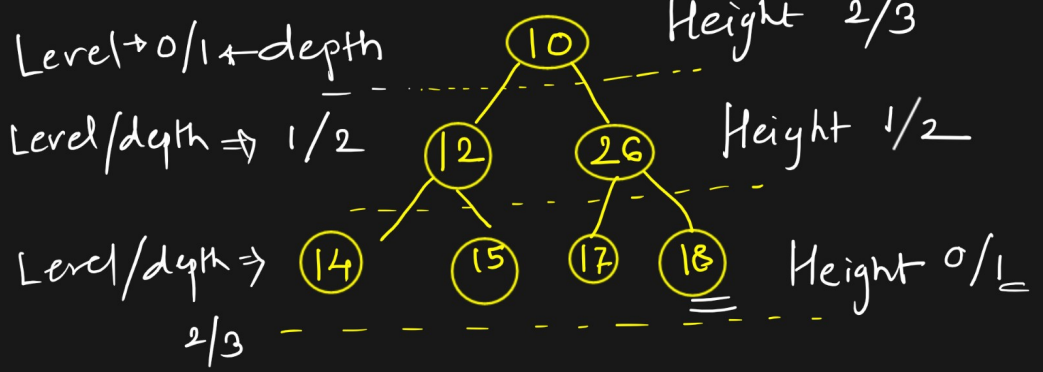
Queue

3) Linked List

Visualization of Tree (Representation)

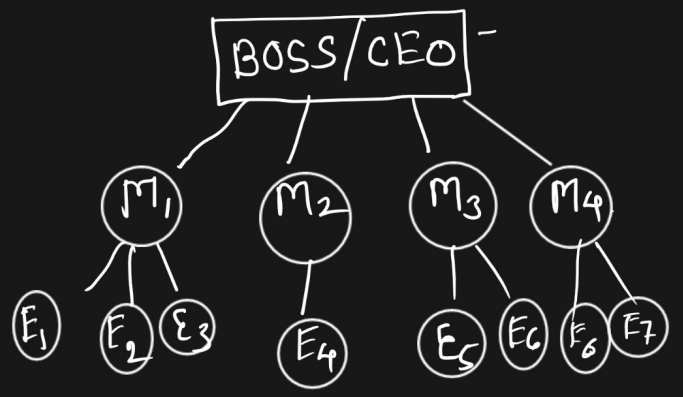


* Terminology in Tree



Application of Tree

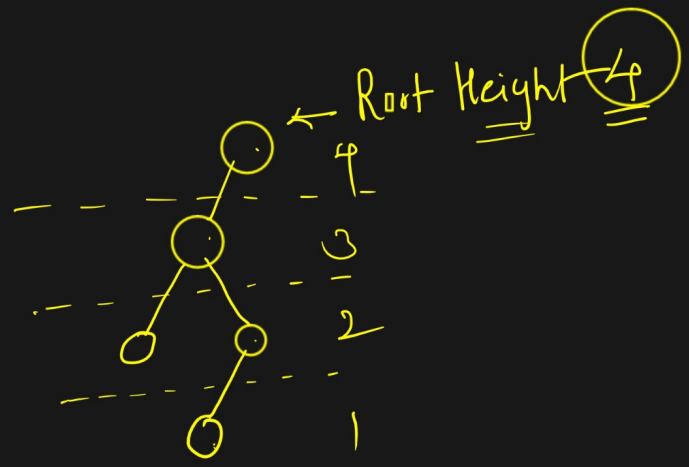
1) Hirarchy Data to store



2) Social Media appln

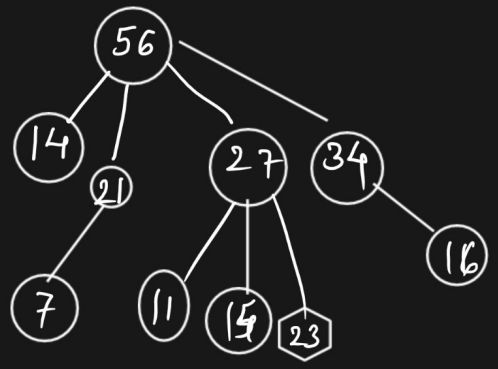
3) Inside the graph for traversing

4) Expression solve



* Types of Tree :-

1) General Tree :-



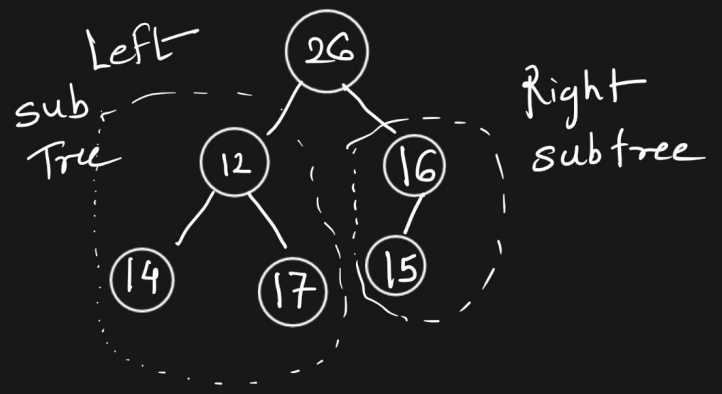
→ No limitation for the child nodes

2) Binary Tree

⇒ Each node have at most 2 child (0, 1, 2)

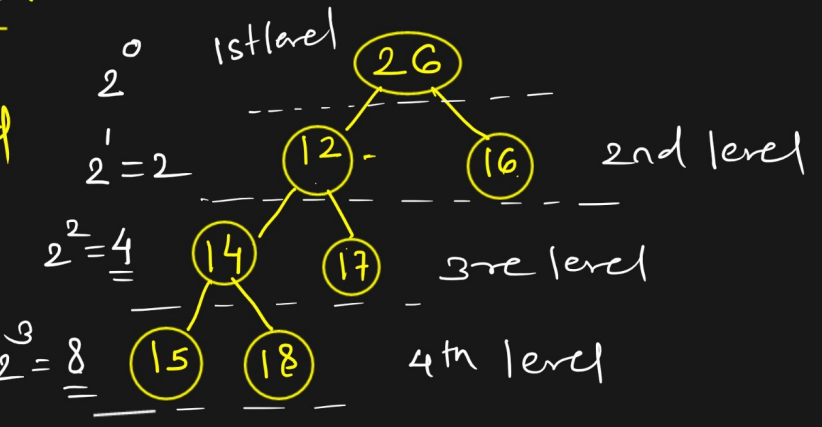
⇒ But the elements are not stored in seq?

26, 12, 14, 17, 16, 15



3) Full Binary Tree / Strict Binary Tree :-

⇒ every node has either 0 or 2 child

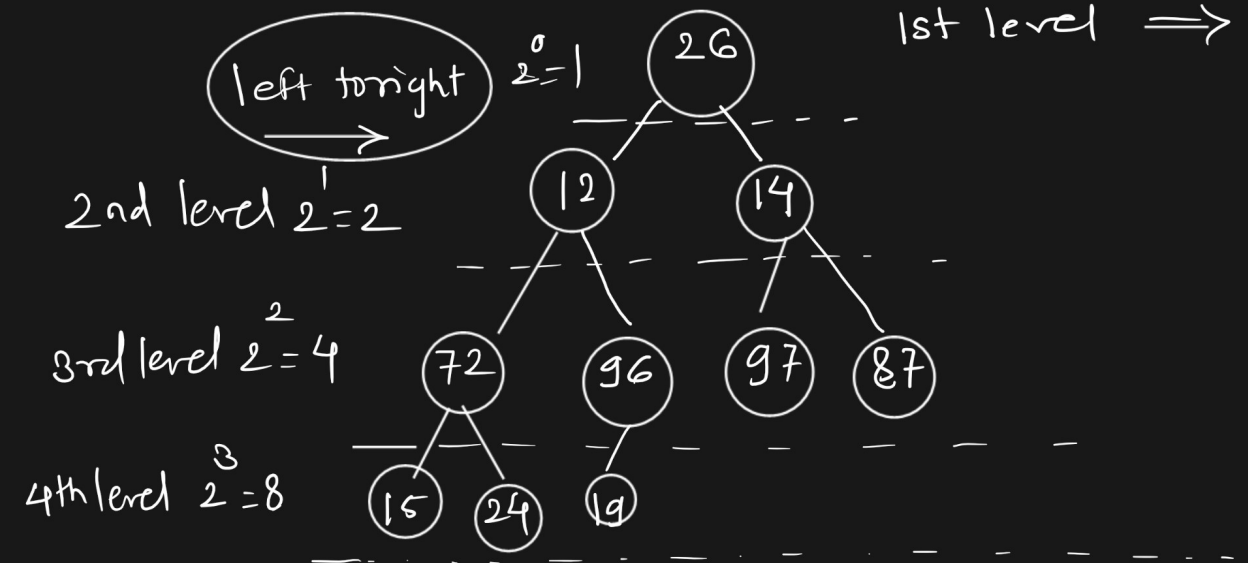


4) Complete Binary Tree :-

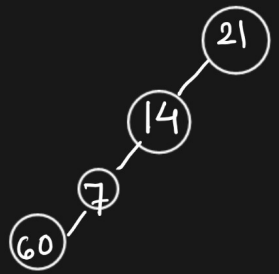
⇒ Every node have either 0 or 1 or 2 child

⇒ But we have to insert in the fashion of left to right and complete the level.

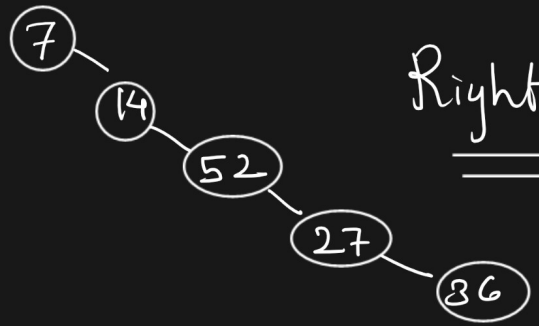
26, 12, 14, 72, 96, 97, 87, 15, 24



5) Skew Binary Tree :-



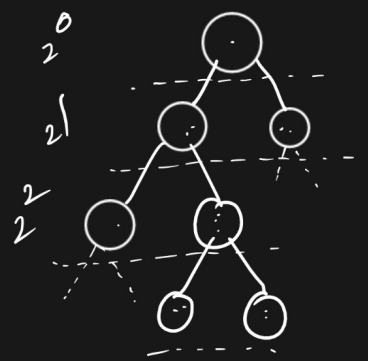
Left Skew Tree



Right Skew Tree

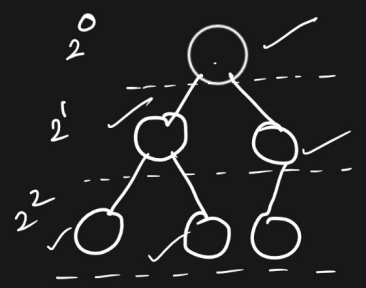
Examples :-

A)



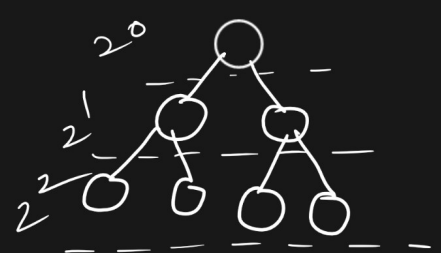
FBT ✓
CBT X

B)



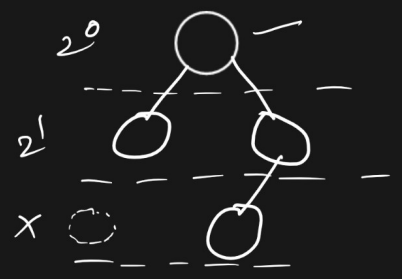
FBT X
CBT ✓

C)



FBT ✓
CBT ✓

D)



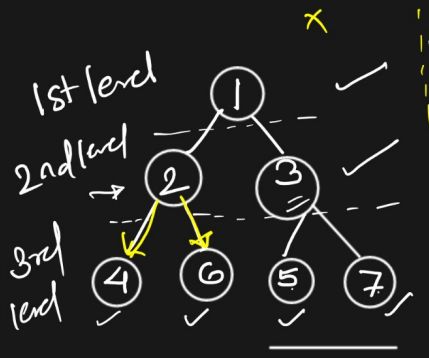
FBT X
CBT X

* Binary Tree Representation :-

A) Array Representation ⇒ Drawback ⇒ memory wastage

B) Linked List Representation ⇒ Always we used this representation

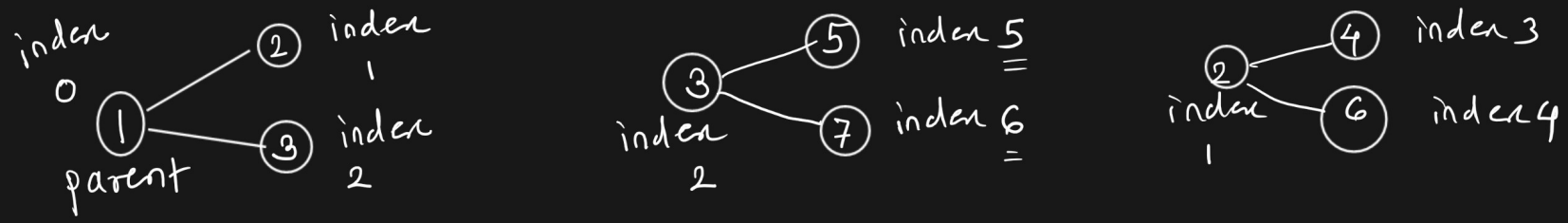
A) Array Representation :-



Values
Index
Siblings

1	2	3	4	6	5	7
0	1	2	3	4	5	6

We insert the de in array likewise from left to right levelwise



Find out child nodes from parent node index as well as find out parent node index from Array.

Values	1	2	3	4	6	5	7
index	0	1	2	3	4	5	6

2 (parent) ⇒ index 1 ⇒ child index ?

child node index {
- left child index = (parent index * 2) + 1
- right child index = (parent index * 2) + 2

e.g. left index = (1 * 2) + 1 = 3 ⇒ index 4

right index = (1 * 2) + 2 = 4 ⇒ index 6

---

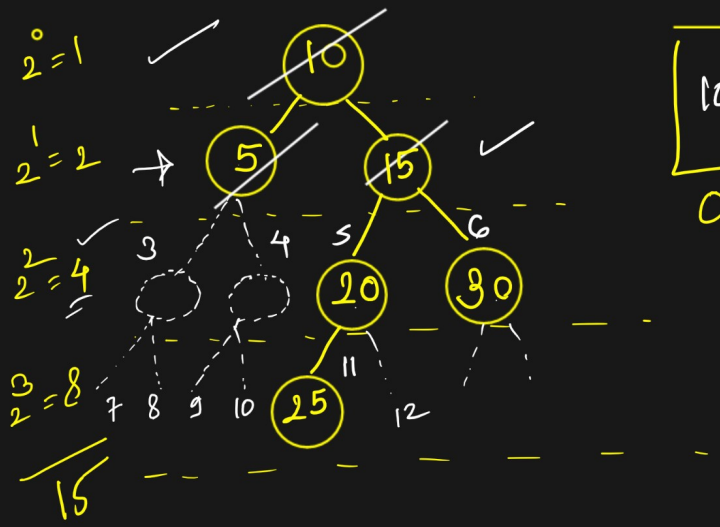
parent index {
- parent index = $\left\lfloor \frac{\text{left index}}{2} \right\rfloor$ → Floor
- parent index = $\left\lfloor \frac{\text{right child index}}{2} - 1 \right\rfloor$

e.g. $\frac{3}{2} = \lfloor 1.5 \rfloor = 1$

$\frac{4}{2} - 1 = 2 - 1 = 1$

Disadvantage of Array Representation:-

Node = 6 Array of size = 15

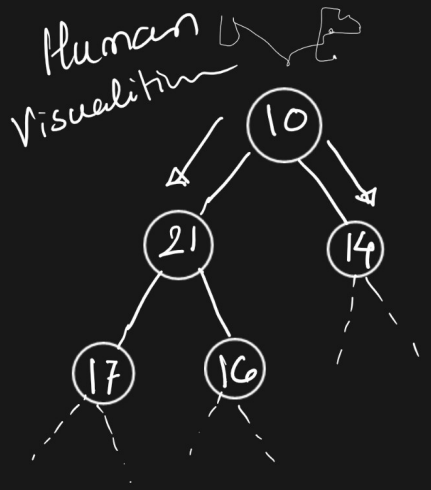


10	5	15	null	null	20	30	null	null	null	null	25	null	null	null
0	1	2	3	4	5	6	7	8	9	10	11	12	13	14

15
− 6
9 — location ⇒ null

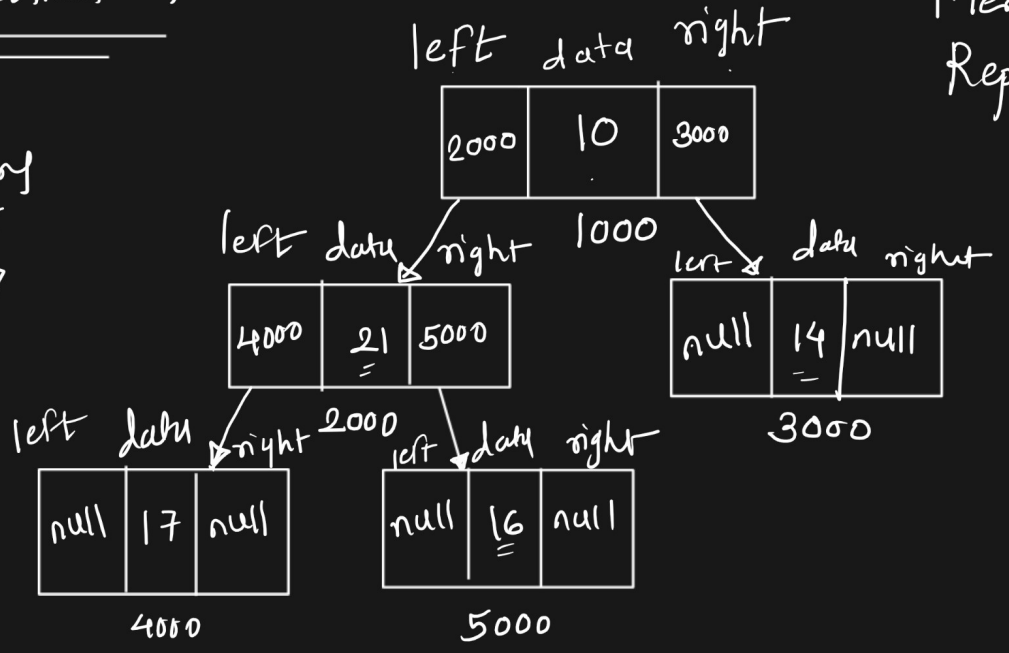
Wastage of memory

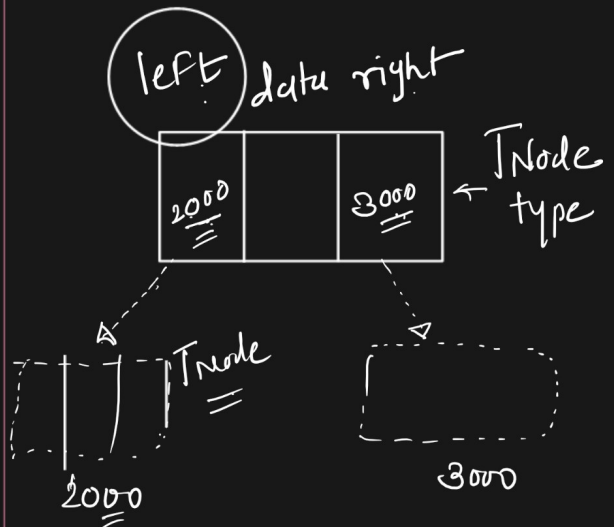
B) Linked List Representation :- chain of nodes



memory ⟹

Memory Representation

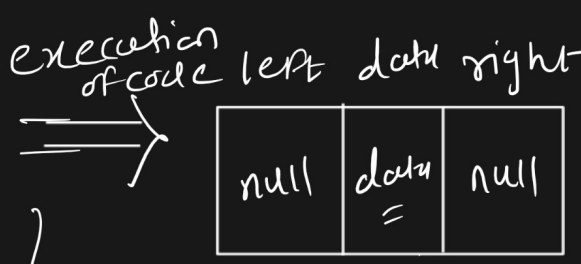





```
class TNode
{
    int data;
    TNode left;
    TNode right;

    public TNode(int data)
    {
        this.data = data;
        left = null;
        right = null;
    }
}
```

after the constructor execution of code



⇒ TNode obj = new TNode(10);